

DSS5201 Data Visualisation

Week 4 Importing and Exploring Data

David CHEW

Semester 2 AY25/26

Last Week: Python Basics

- Introduction to NumPy
- Introduction to pandas
- Loops (Repeated Executions)
- Functions
- Writing Readable Code

We learnt how to create data frames from scratch and read data from pydataset.

This Week: Importing Data

This week, we will learn how to import data from external sources.

- Plain Text Files
- CSV Files
- Excel Files
- JSON Files
- Data from the Web

We will also go through some basic ways to clean and explore data.

Prerequisites

File Location



An important prerequisite of loading data is to **point to the location** at which the files are stored.

Two questions:

1. Where am I?
2. Where are my data?

Q1. Where am I?

This question addresses the notion of the **current working directory**.

- This is where Python looks for files and saves outputs by default.
- `getcwd()` from the `os` standard library returns the **full path** of our current working directory.

```
1 import os
2 cwd = os.getcwd()
3 cwd
```

- For instance, on my own machine, my working directory is
`/Users/dshchew/Documents/dss5201/src`

Q2. Where are my data?

Data are not necessarily stored at the current working directory.

- **Relative path:** The address of a file relative to the current working directory.
 - Access files directly in the current path.
 - Use two dots (`..`) to denote “one level up in the directory hierarchy”.

Use relative path in all code you write. This allows you to share your work easily with others.

File Structure

Follow this structure strictly:

```
1 DSS5201
2   |--- data           # Data files
3       |---- tourist.csv
4       |---- ...
5   |--- src           # Scripts, notebooks, rendered HTMLs
6       |---- lecture_wk4.ipynb
7       |---- ...
```

- A main folder named **DSS5201**.
 - A sub-folder named **src** to store all Python scripts and notebooks.
 - A sub-folder named **data** to store all data sets.

Important!!

The **src** and **data** folders should be at the same level within **DSS5201**.

Accessing Files from the data sub-folder

```
1 DSS5201
2 |--- data                # Data files
3 |---- tourist.csv
4 |---- ...
5 |--- src                # Scripts, notebooks, rendered HTMLs
6 |---- lecture_wk4.ipynb
7 |---- ...
```

- The current working directory is set to `src`.
- To access `tourist.csv`, we use the path `../data/tourist.csv`
- What this means:
 - `..` tells Python to move one level up from the current working directory,
 - go into the `data` folder, and
 - open `tourist.csv` inside it.

Data Sets This Week

Download the following from Canvas and store them in the **data** folder.

- wk4_text_01.txt
- wk4_nba.txt
- wk4_earnings.csv
- wk4_excel_01.xlsx
- wk4_injuries.xlsx
- wk4_json_01.json
- wk4_json_02.json

Importing Packages

In your notebook, create a code cell at the beginning and import the following packages.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import json
5 import requests
```

Recall that

- NumPy is for numerical computing.
- pandas is for data import, manipulation, and basic analysis/visualisations.
- Matplotlib supports basic visualisations.

Plain Text File

Plain Text File

Let's first consider a simple file format, the tab-delimited file.

The `pandas` function to read in a text file is `read_table()`, with the main arguments:

- `path`: A string indicating file location.
- `header`: The row index containing column names.
- `sep`: The delimiter used to split columns. The default is a tab (`\t`).

```
1 data1 = pd.read_table('../data/wk4_text_01.txt',  
2                       header = 0,  
3                       sep = '\t')  
4 data1
```

	height	sex	earnings
0	189	male	50000
1	166	female	60000

Plain Text File

- Sometimes the data do not have column names.
- We might have NaN's in the data too.

This is the case with the file `wk4_nba.txt`:

```
1 Avery Bradley    Boston Celtics  7730337
2 Jae Crowder Boston Celtics  6796117
3 Lance Thomas    New York Knicks NA
4 Stephen Curry   Golden State Warriors  11370786
```



Plain Text File

We use

- `header = None` to specify that there is no column names,
- `name` to provide a list of column names, and
- `na_values` to specify value(s) to be replaced with `NaN`.

```
1 data2 = pd.read_table('../data/wk4_nba.txt', header = None,  
2                        sep = '\\t', names = ['name', 'team', 'salary'],  
3                        na_values = 'NA')  
4 data2
```

	name	team	salary
0	Avery Bradley	Boston Celtics	7730337.0
1	Jae Crowder	Boston Celtics	6796117.0
2	Lance Thomas	New York Knicks	NaN
3	Stephen Curry	Golden State Warriors	11370786.0

CSV File

CSV File

We now consider another simple file format – the comma-separated values (CSV) file.

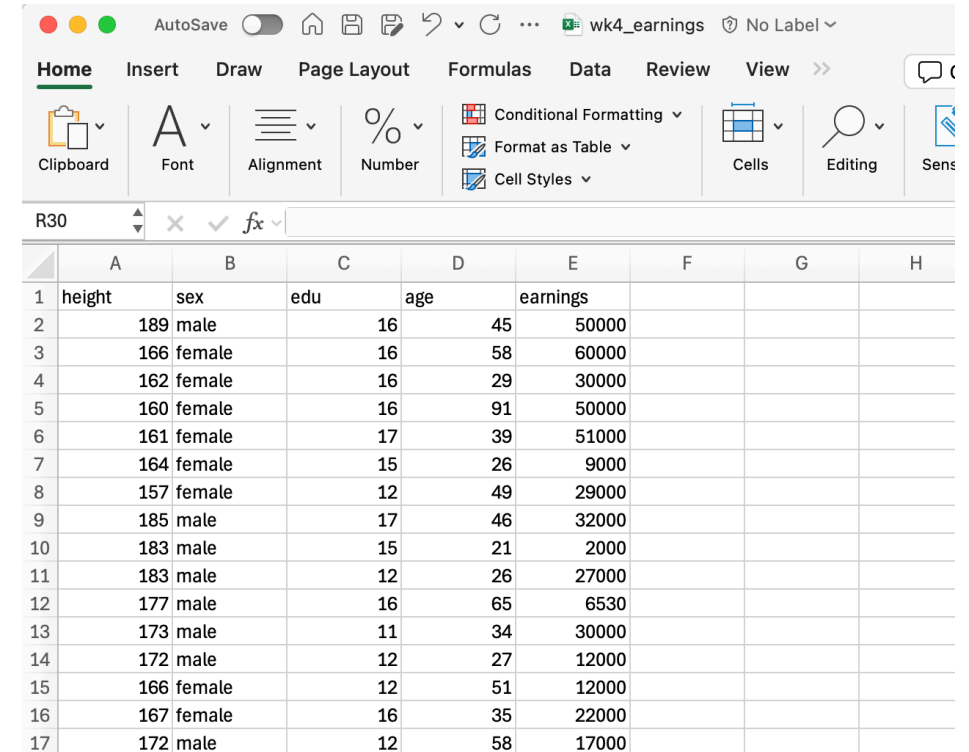
These files are in fact also text files, with

- An optional header, and
- Each observation separated by commas within each row.

CSV File: How does it look like?

A CSV file displayed in any text editor: The same file opened in MS Excel:

```
1 height,sex,edu,age,earnings
2 189,male,16,45,50000
3 166,female,16,58,60000
4 162,female,16,29,30000
5 160,female,16,91,50000
6 161,female,17,39,51000
7 164,female,15,26,9000
8 157,female,12,49,29000
9 185,male,17,46,32000
10 183,male,15,21,2000
11 183,male,12,26,27000
12 177,male,16,65,6530
13 173,male,11,34,30000
14 172,male,12,27,12000
15 166,female,12,51,12000
16 167,female,16,35,22000
17 172,male,12,58,17000
18 174,female,14,29,40000
19 177,male,13,44,44000
20 163,female,12,55,7000
```



The screenshot shows the MS Excel interface with the CSV data imported as a table. The table has 5 columns: height, sex, edu, age, and earnings. The data is displayed in rows 1 through 17. The Excel ribbon is visible at the top, showing the Home tab with options for Clipboard, Font, Alignment, Number, Conditional Formatting, Format as Table, and Cell Styles. The status bar at the bottom indicates the active cell is R30.

	A	B	C	D	E	F	G	H
1	height	sex	edu	age	earnings			
2	189	male		16	45	50000		
3	166	female		16	58	60000		
4	162	female		16	29	30000		
5	160	female		16	91	50000		
6	161	female		17	39	51000		
7	164	female		15	26	9000		
8	157	female		12	49	29000		
9	185	male		17	46	32000		
10	183	male		15	21	2000		
11	183	male		12	26	27000		
12	177	male		16	65	6530		
13	173	male		11	34	30000		
14	172	male		12	27	12000		
15	166	female		12	51	12000		
16	167	female		16	35	22000		
17	172	male		12	58	17000		

Reading a CSV File

The `pandas` function to read a CSV file is `read_csv()`. The main arguments are:

- `file`: The file path.
- `header`: The row index containing column labels (zero-indexed).
- `names`: A list of column names to apply.
- `skiprows`: Number of lines at the beginning to skip.
- `na_values`: The strings to be recognized as missing values.

Example: Earnings Data

The file `wk4_earnings.csv` contains information on 1192 individuals.

Have a look at the data. It contains 5 columns. The header is in the first row.

```
1 df = pd.read_csv('../data/wk4_earnings.csv', header = 0)
2 df.head()
```

	height	sex	edu	age	earnings
0	189	male	16	45	50000
1	166	female	16	58	60000
2	162	female	16	29	30000
3	160	female	16	91	50000
4	161	female	17	39	51000

Data Checks

1. How many rows and columns are in the data frame?

```
1 df.shape
```



```
(1192, 5)
```

2. What is the data type of each column?

```
1 df.dtypes
```



```
height      int64
sex          str
edu          int64
age          int64
earnings     int64
dtype: object
```

Data Checks

3. `sex` has been read in as strings (`object`).

- Let's check the distinct values that have been read in.

```
1 df['sex'].unique()
```

```
<StringArray>
```

```
['male', 'female', 'Female']
```

```
Length: 3, dtype: str
```

 **The categories are inconsistent!**

Data Checks

We can standardize them by converting the values to lower case:

- `str` is the string accessor in `pandas`.
- It allows us to apply the `.lower()` method to each element in the column.

```
1 df['sex'] = df['sex'].str.lower()  
2 df['sex'].unique()
```

<StringArray>

['male', 'female']

Length: 2, dtype: str

Data Checks

We next convert `sex` into categorical data.

- It helps to save memory and make downstream operations more convenient.

```
1 df['sex'] = pd.Categorical(df['sex'])  
2 df['sex'].dtype
```

CategoricalDtype(categories=['female', 'male'], ordered=False, categories_dtype=str)

- Alternatively, use `.astype('category')`.

```
1 df['sex'] = df['sex'].astype('category')
```


Data Checks

4. Are there missing values in the data?

- We can use `.isna()` to detect missing value.
- Count the number of missing values by column: `.isna().sum()`.

```
1 df.isna().sum()
```

```
height      0  
sex          0  
edu         0  
age         0  
earnings    0  
dtype: int64
```

Excel File

Excel File

Microsoft Excel files come in different `formats and extensions`:

- For `.xlsx` files, `pandas` uses the `openpyxl` engine.
- For older `.xls` files, it uses the `xlrd` engine.

As long as these packages are installed, `pd.read_excel()` will look at the file extensions and automatically use the right engine.

Excel File

Let's read in our first Excel file, `wk4_excel_01.xlsx`.

```
1 df_xlsx = pd.read_excel('../data/wk4_excel_01.xlsx')
2 df_xlsx.head()
```

	Table 1	Unnamed: 1	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	NaN	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	a	1.0	m

Excel File

What's wrong with the data?

Data seem to be 'floating' in the middle of the worksheet.

In this case, `pd.read_excel()` needs a little help.

- Use `skiprows` and `usecols` to tell Python the location of the data.
- Specify the column names with `names`.

```
1 df_xlsx = pd.read_excel('../data/wk4_excel_01.xlsx',
2                           skiprows = 5, usecols = 'C:E', header = None,
3                           names = ['col1', 'col2', 'col3'])
4 df_xlsx.head()
```

	col1	col2	col3
0	a	1	m
1	b	2	m
2	c	3	m

Excel File: Workplace Injuries

The Excel file `wk4_injuries.xlsx` contains data on selected types of workplace injuries from 2019 to 2022, obtained from the Ministry of Manpower. By default, the function reads in the first spreadsheet in the file.

```
1 injuries = pd.read_excel('../data/wk4_injuries.xlsx')
2 injuries.head()
```

	Type	2019	2020	2021	2022	2023	2024
0	Crushing, fractures and dislocations	3107.0	2577.0	2950.0	2759.0	2953	2798
1	Cuts and Bruises	4500.0	3895.0	4263.0	4333.0	4373	3950
2	Sprains & Strains	1982.0	1791.0	1829.0	1778.0	1807	1668
3	Others	2418.0	1675.0	2100.0	2022.0	n.a	n.a
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Excel File: Workplace Injuries

To read in the correct data, we should specify

- an appropriate range of cells, and
- how missing values are to be represented.

```
1 injuries = pd.read_excel('../data/wk4_injuries.xlsx',
2                           nrows = 4, usecols = 'A:G',
3                           na_values = 'n.a')
4 injuries.head()
```

	Type	2019	2020	2021	2022	2023	2024
0	Crushing, fractures and dislocations	3107	2577	2950	2759	2953.0	2798.0
1	Cuts and Bruises	4500	3895	4263	4333	4373.0	3950.0
2	Sprains & Strains	1982	1791	1829	1778	1807.0	1668.0
3	Others	2418	1675	2100	2022	NaN	NaN

Excel File: Multiple Worksheets

`pd.ExcelFile()` opens the file and stores the spreadsheet(s) in memory.

```
1 xls = pd.ExcelFile('../data/wk4_injuries.xlsx')
2 xls.sheet_names
```

```
['Type', 'Industry']
```

The sheets can then be accessed with `.parse()`.

```
1 industries = xls.parse(sheet_name = 'Industry',
2                          nrows = 3, usecols = 'A:G',
3                          na_values = 'n.a')
4 industries.head()
```

	Industry	2019	2020	2021	2022	2023	2024
0	Manufacturing	143.0	110.0	140.0	122.0	150	123
1	Construction	135.0	74.0	127.0	157.0	149	146
2	Work-related Traffic Accidents	NaN	NaN	NaN	NaN	2	3

Excel File: Multiple Worksheets

We could have also accessed the sheet directly:

```
1 industries = pd.read_excel('../data/wk4_injuries.xlsx',
2                             sheet_name = 'Industry',
3                             nrows = 3, usecols = 'A:G',
4                             na_values = 'n.a')
5 industries.head()
```

	Industry	2019	2020	2021	2022	2023	2024
0	Manufacturing	143.0	110.0	140.0	122.0	150	123
1	Construction	135.0	74.0	127.0	157.0	149	146
2	Work-related Traffic Accidents	NaN	NaN	NaN	NaN	2	3

Data Exploration

Data Exploration

There are two main ways of summarizing data:

1. **Numerical** summary
2. **Graphical** summary

Numerical summaries include:

- Basic information about the data set, i.e., data checks.
- Measures of central tendency, e.g., mean, median.
- Measures of spread, e.g., standard deviation, interquartile range (IQR), range.

Numerical Summary: Earnings Data

We revisit the Earnings data and generate numerical summaries for `earnings`:

```
1 df = pd.read_csv('../data/wk4_earnings.csv', header = 0)
2 df['sex'] = df['sex'].str.lower()
3 df['earnings'].describe()
```

```
count      1192.000000
mean       23154.773490
std        19472.296925
min         200.000000
25%        10000.000000
50%        20000.000000
75%        30000.000000
max        200000.000000
```

```
Name: earnings, dtype: float64
```

Numerical Summary: Earnings Data

From the output, we can understand that we have 1192 observations.

- Earnings range from 200 to 2000000 dollars.
- The distribution of earnings is skewed to the right (median < mean).

However, summaries of a single variable are not very useful, since we do not have a basis for comparison.

Numerical Summary by group: Earnings Data

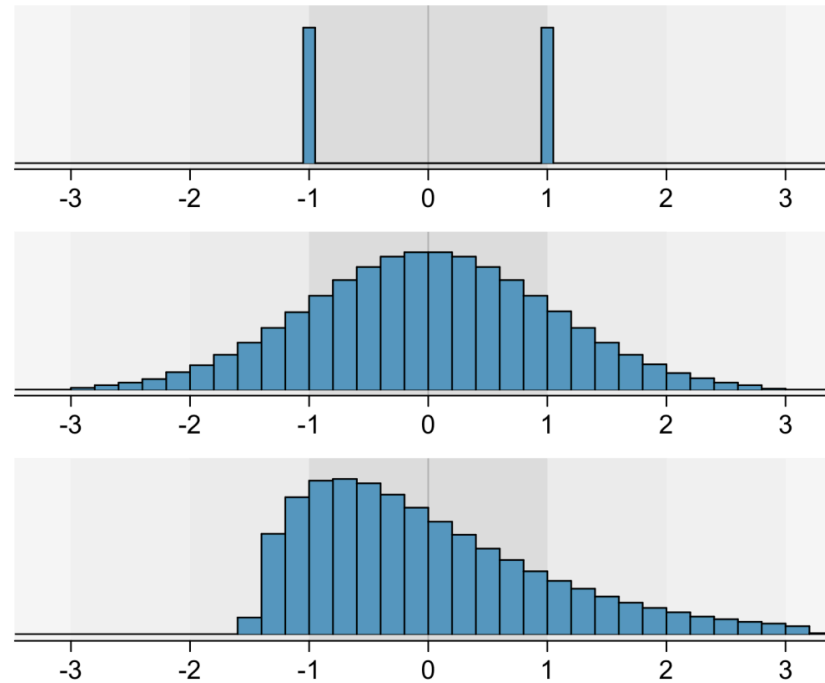
It is more interesting to explore how a variable varies with one or more variables.

We can compute **group statistics** for `earnings`:

```
1 df.groupby('sex')['earnings'].describe()
```

	count	mean	std	min	25%	50%	75%	max
sex								
female	687.0	18280.195051	14259.932378	200.0	9000.0	15000.0	25000.0	123000.0
male	505.0	29786.130693	23297.816034	1000.0	16000.0	25000.0	36000.0	200000.0

Graphical Summary



While numerical summaries provide us some basic information, they also leave out a lot.

- Possible to have a wrong mental idea of the data from numerical summaries alone.
- We also need **graphical summaries**.

Graphical Summary: Earnings Data

Let's visualize `earnings` in this data set.

- `pandas` relies on `Matplotlib` as its default plotting backend.
- That is why we loaded the `Matplotlib` library at the very beginning.

We can use a **histogram** to visualize **quantitative** variables like `earnings`.

- Uses bars to portray the frequency of possible outcomes for a quantitative variable.

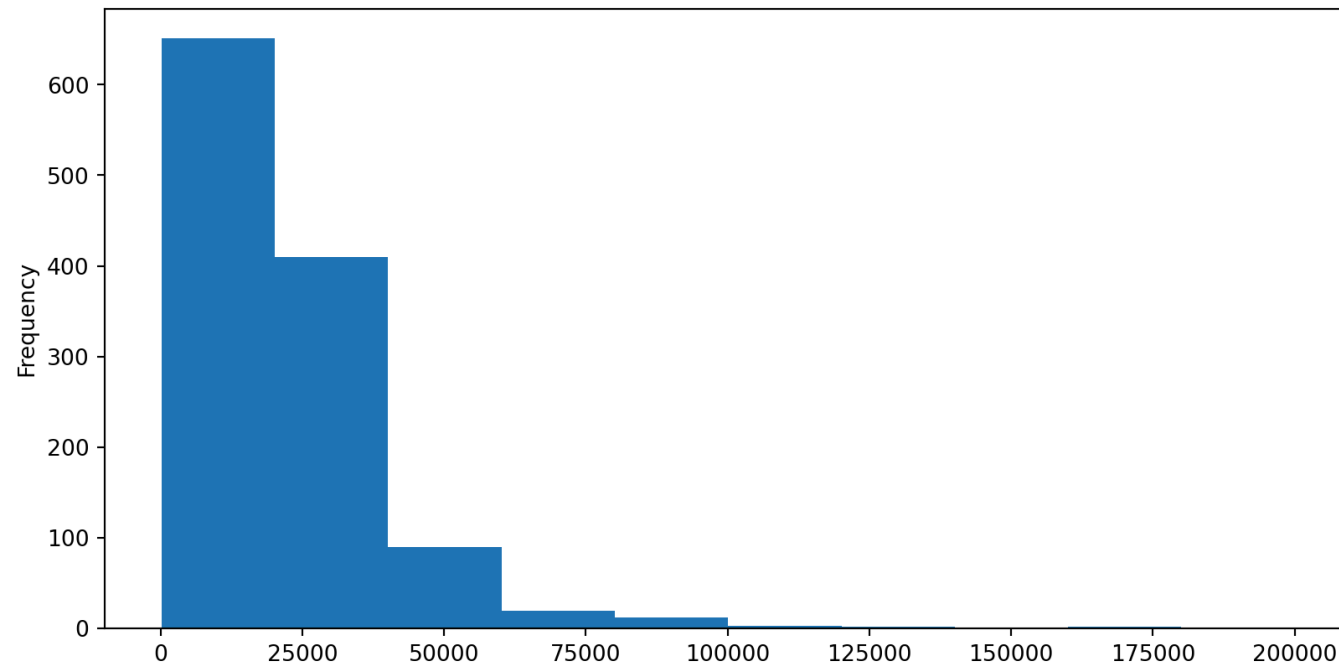
Graphical Summary: Histogram

When we create a histogram, here are some things we look for:

- What is the overall pattern?
 - Do the data cluster together, or is there a gap such that one or more observations deviate from the rest?
 - Are there any suspected outliers?
- Do the data have a single peak?
- Is the distribution symmetric or skewed?

Histogram: Earnings Data

```
1 df['earnings'].plot(kind = 'hist')
```



Histogram: Earnings Data

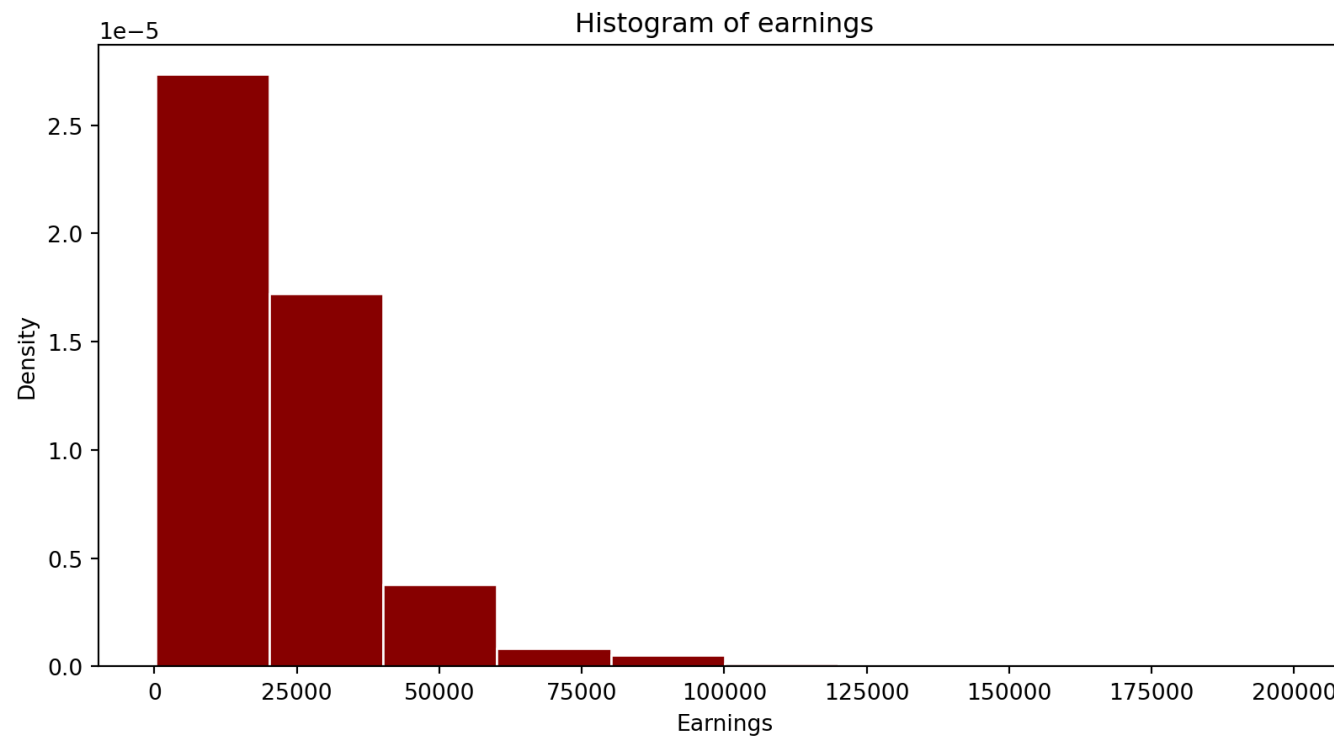
```
1 df['earnings'].plot(kind = 'hist')
```



- `kind = 'hist'` specifies that we want to create a histogram.
- By default, the height represents frequencies. This can be modified with `density = True`.
- Additional arguments we can specify:
 - `title, xlabel, ylabel`
 - `color`
 - `edgecolor`

Histogram: Earnings Data

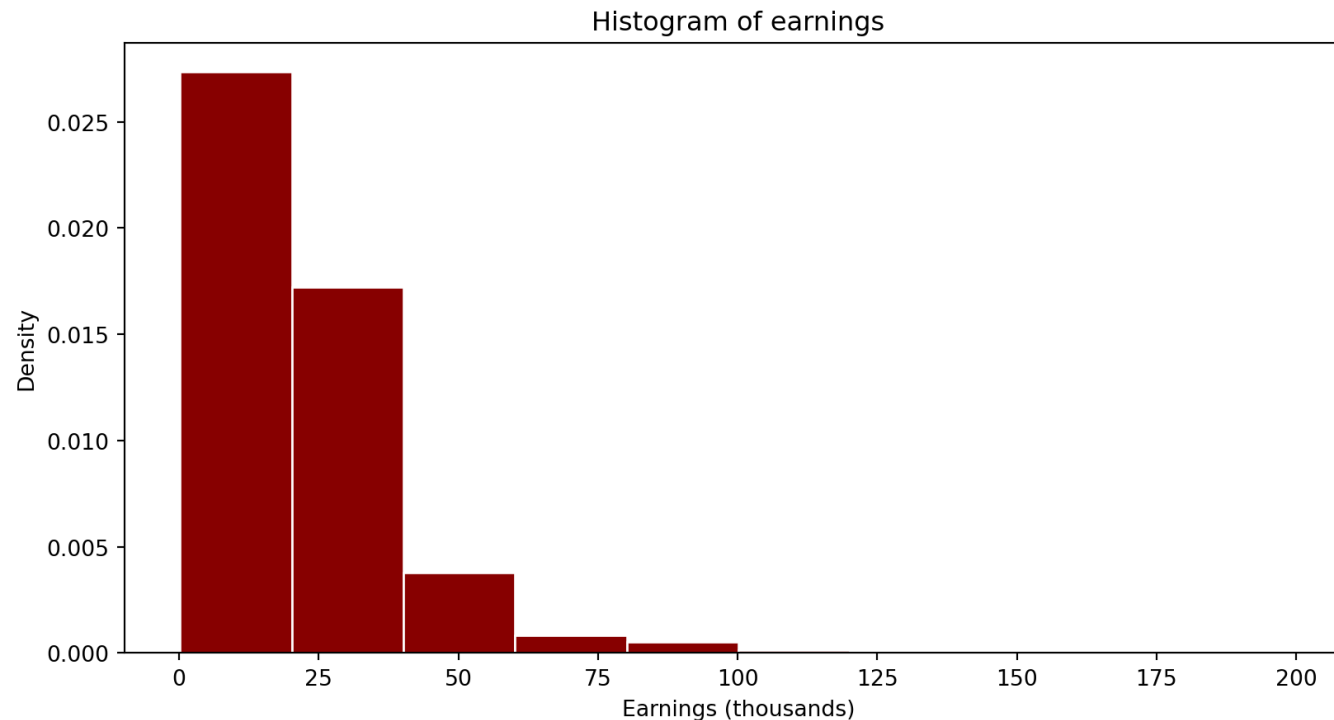
```
1 df['earnings'].plot(kind = 'hist', density = True,  
2                        title = 'Histogram of earnings',  
3                        xlabel = 'Earnings', ylabel = 'Density',  
4                        color = 'darkred', edgecolor = 'white')
```



Histogram: Earnings Data

Re-scale `earnings` by 1000 so the x-axis unit is in thousands of dollars.

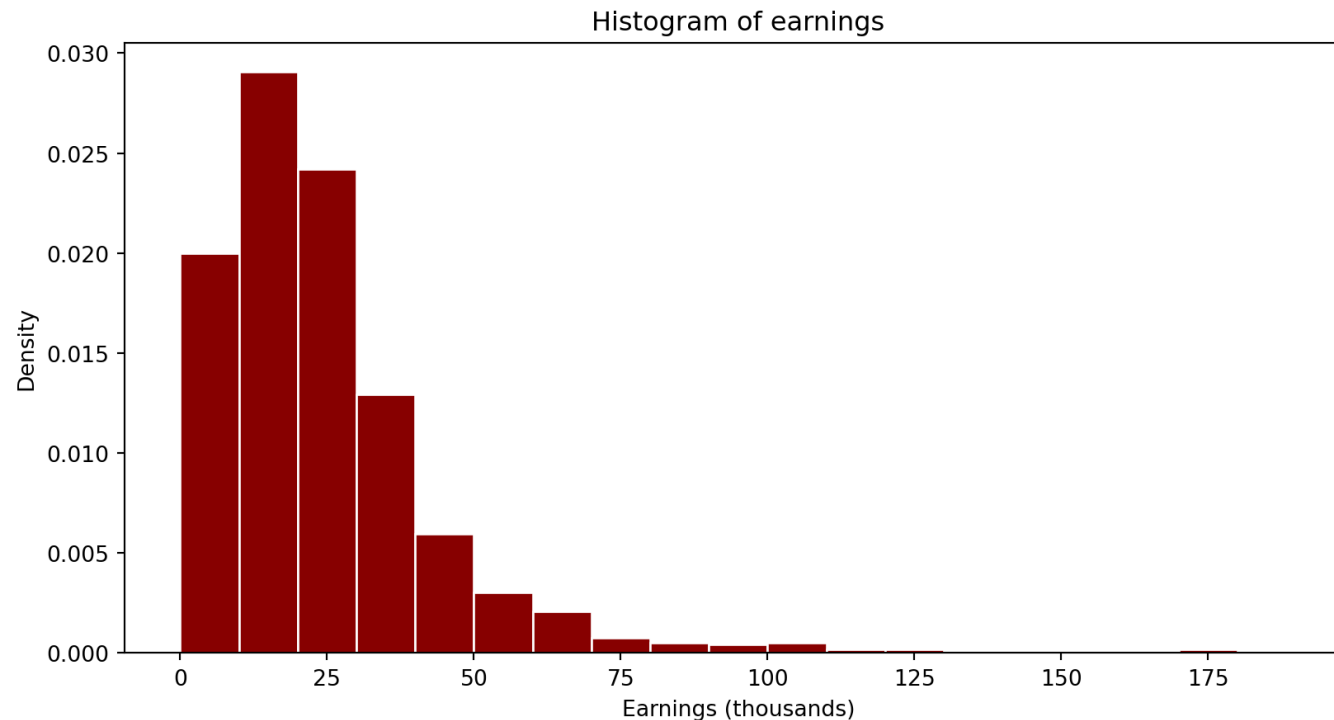
```
1 df['earn_k'] = df['earnings'] / 1000
2 df['earn_k'].plot(kind = 'hist', density = True,
3                   title = 'Histogram of earnings',
4                   xlabel = 'Earnings (thousands)',
5                   ylabel = 'Density',
6                   color = 'darkred', edgecolor = 'white')
```



Histogram: Earnings Data

Modify bin widths to 10000 to get finer details of the distribution.

```
1 df['earn_k'].plot(kind = 'hist', density = True,  
2                     bins = range(0, 200, 10),  
3                     title = 'Histogram of earnings',  
4                     xlabel = 'Earnings (thousands)',  
5                     ylabel = 'Density',  
6                     color = 'darkred', edgecolor = 'white')
```



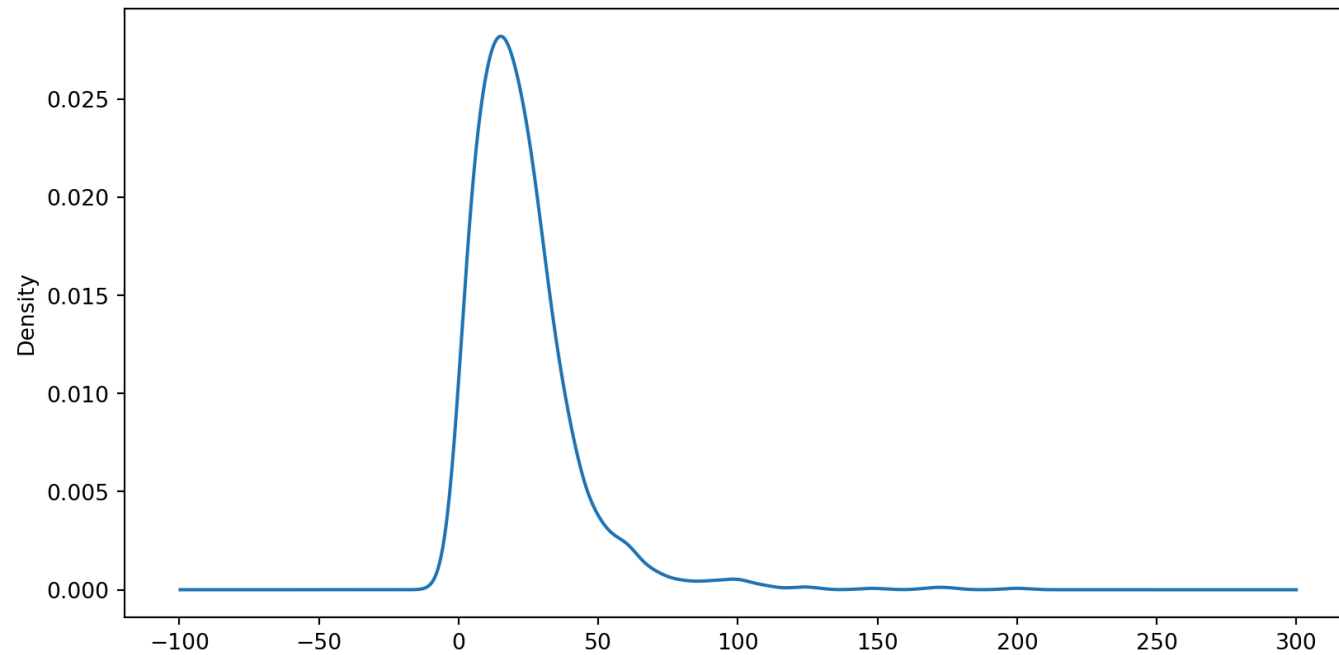
Graphical Summary: Density Plot

Histograms are not perfect.

- We have to experiment with bin widths since this could mask details about the data.
- An alternative is the **kernel density plot** – by smoothing the heights of the rectangle bars.
 - `pandas` relies on the `SciPy` library to compute the kernel density estimates.
 - Make sure to install the `SciPy` library in your virtual environment.

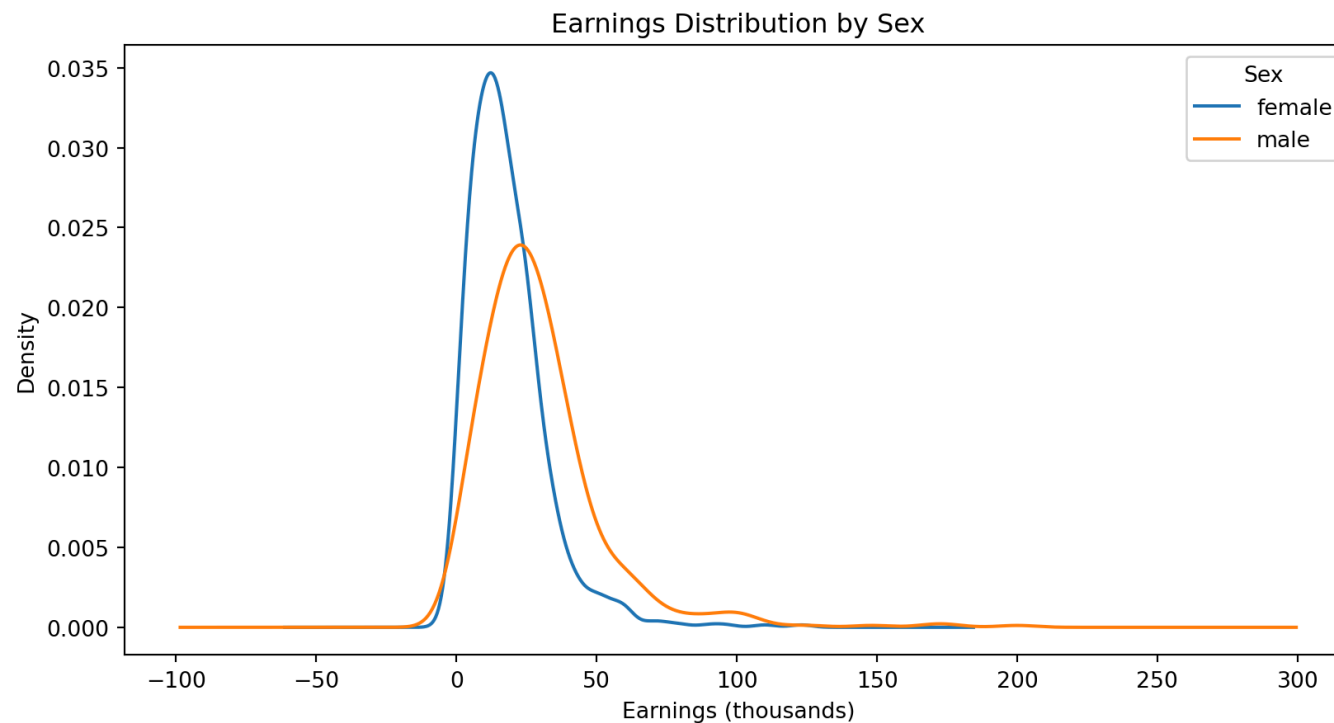
Density Plot: Earnings Data

```
1 df['earn_k'].plot(kind = 'kde')
```



Density Plot by group: Earnings Data

```
1 for subgroup, subset in df.groupby('sex'):
2     subset['earn_k'].plot(kind = 'kde', label = subgroup)
3 plt.legend(title = 'Sex')
4 plt.xlabel('Earnings (thousands)')
5 plt.ylabel('Density')
6 plt.title('Earnings Distribution by Sex')
7 plt.show()
```



Density Plot by group: Earnings Data

```
1 for subgroup, subset in df.groupby('sex'):
2     subset['earn_k'].plot(kind = 'kde', label = subgroup)
3 plt.legend(title = 'Sex')
4 plt.xlabel('Earnings (thousands)')
5 plt.ylabel('Density')
6 plt.title('Earnings Distribution by Sex')
7 plt.show()
```

`for subgroup, subset in ...` loops over each group.

- `subgroup` is the group name (`female` or `male`)
- `subset` is the sub-data frame that contains each subgroup.

The rest are `matplotlib` functions that set the legend, axis labels, title, and then display the plot.

pandas Plotting Functions

The following methods are equivalent in creating plots with `pandas`.

- `#1` below specifies the plot type with `kind`.
- `#2` below uses the dot notation for each plot type.

```
1 # 1
2 df['earn_k'].plot(kind = 'kde')
3
4 # 2
5 df['earn_k'].plot.kde()
```



More Data Exploration: Binning

The variable `earnings` ranges from 200 to 200000 dollars.

Let's create earning groups based on percentiles.

```
1 df['earnings'].describe()
```

```
count      1192.000000
mean       23154.773490
std        19472.296925
min         200.000000
25%        10000.000000
50%        20000.000000
75%        30000.000000
max        200000.000000
```

```
Name: earnings, dtype: float64
```

Binning by Quantiles: Earnings Data

We can split data into quantile-based bins with `qcut()`.

```
1 groups = ['Q1', 'Q2', 'Q3', 'Q4']
2 df['earn_grp'] = pd.qcut(df['earnings'], q = 4, labels = groups)
3 df['earn_grp'].value_counts(sort = False)
```

earn_grp

Q1	300
Q2	350
Q3	276
Q4	266

Name: count, dtype: int64

Binning by Intervals: Earnings Data

We can also split data into interval-based bins with `cut()`.

It uses fixed numeric boundaries.

```
1 groups = [0, 200, 10000, 30000, 200000]
2 df['earn_bins'] = pd.cut(df['earnings'], bins = groups)
3 df['earn_bins'].value_counts(sort = False)
```

earn_bins

(0, 200]	2
(200, 10000]	298
(10000, 30000]	626
(30000, 200000]	266

Name: count, dtype: int64

Contingency Table by Counts: Earnings Data

Lastly, `crosstab()` creates a contingency table.

Frequency counts for each combination of `earnings_grp` and `sex`.

```
1 pd.crosstab(df['earn_grp'], df['sex'])
```

	sex	
	female	male
earn_grp		
Q1	228	72
Q2	226	124
Q3	145	131
Q4	88	178

Contingency Table by Proportions: Earnings Data

We can also display row proportions (percentages within each `earn_grp`):

```
1 pd.crosstab(df['earn_grp'], df['sex'], normalize = 'index')
```

	sex	female	male
earn_grp			
Q1		0.760000	0.240000
Q2		0.645714	0.354286
Q3		0.525362	0.474638
Q4		0.330827	0.669173

Try out the other options too: `columns`, `all`.

JSON File

JSON

JSON (JavaScript Object Notation) is a standard **text-based format** for storing structured data.

- A popular format for data interchange on the internet.
- The full description of the format can be found at <http://www.json.org/>.
- The syntax is easy for humans to read and write, and for computers to parse and generate.

JSON Description

```
[
  {
    "first_name": "John",
    "last_name": "Smith",
    "is_alive": true,
    "age": 27,
    "address": {
      "city": "New York",
      "state": "NY"
    },
    "children": [
      "Catherine",
      "Thomas",
      "Trevor"
    ]
  }
]
```

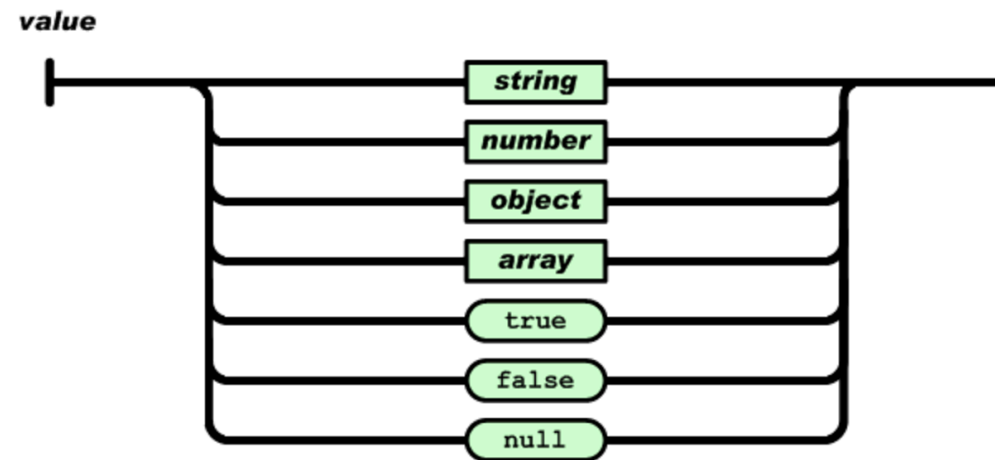
JSON is built on three building blocks:

- An **array** is an ordered list of values.
- An **object** is an un-ordered collection of key-value pairs.
- A **value** is a basic unit of data, can be simple as a string, number, boolean, or complex as an object or an array.

By repeatedly nesting these structures, JSON can represent very complex data.

JSON Value

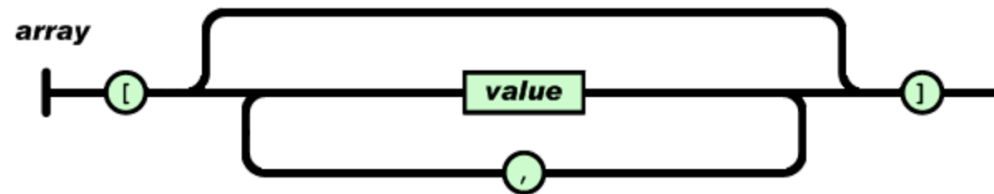
A **value** can be a string (in double quotes), a number, an array, an object, `true`, `false` or `null`.



JSON Array

An **array** is an ordered collection of values, represented as Python lists.

- Surrounded by brackets `[]`
- Values are separated by a comma `,`



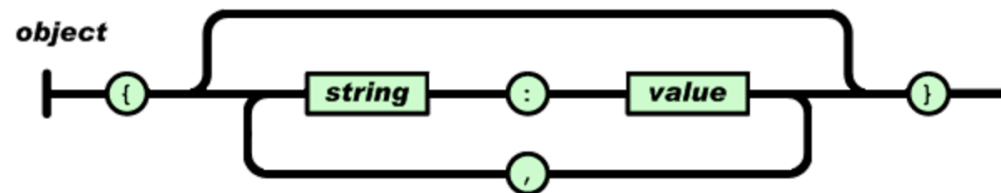
Examples

- `["Catherine", "Thomas", "Trevor"]` is an array with three elements, all are strings.
- `["Hello", 3, 7]` is an array with mixed types.

JSON Object

An **object** is an un-ordered set of key-value pairs, represented as dictionaries.

- Surrounded by braces `{}`
- Each key is followed by a colon `:`
- The key-value pairs are separated by a comma `,`



Examples

- `{"firstname": "John"}` is a valid JSON object with a single key-value pair.
- `{"firstname": "John", "lastname": "Smith"}` is an object with two key-value pairs.

JSON File: Example 1

Examine the structure of `wk4_json_01.json`.

- A JSON array with 1 object, which contains 7 key-value pairs.

When the JSON structure is flat and simple, we can use `pd.read_json()`.

- It reads the file as a data frame with 1 row and 7 columns.

```
1 json1 = pd.read_json('../data/wk4_json_01.json')
2 json1
```

	first_name	last_name	is_alive	age	city	state	children
0	John	Smith	True	27	New York	NY	Catherine, Thomas, Trevor

JSON File: Example 2

The structure of `wk4_json_02.json` is more complex.

- A JSON array with 1 object with 6 key-value pairs.
 - Two of the values has nested structures.
- The simple `pd.read_json()` function **will not work well**. 🚫

```
1 json2 = pd.read_json('../data/wk4_json_02.json')
2 json2
```

	first_name	last_name	is_alive	age	address	children
0	John	Smith	True	27	{'city': 'New York', 'state': 'NY'}	[Catherine, Thomas, Trevor]

JSON File: Example 2

- To read in JSON file with nested structure, we need the `json` standard library.
- Open the file, then load it as a Python list.

```
1 import json
2 with open('../data/wk4_json_02.json') as file:
3     json2 = json.load(file)
4 json2
```

```
[{'first_name': 'John',
  'last_name': 'Smith',
  'is_alive': True,
  'age': 27,
  'address': {'city': 'New York', 'state': 'NY'},
  'children': ['Catherine', 'Thomas', 'Trevor']}]
```

JSON File: Example 2

- Then we can flatten the nested object into columns:

```
1 df = pd.json_normalize(data = json2)
2 df
```

	first_name	last_name	is_alive	age	children	address.city	address.state
0	John	Smith	True	27	[Catherine, Thomas, Trevor]	New York	NY

- To flatten the nested arrays into rows:

```
1 df = df.explode('children', ignore_index = True)
2 df
```

	first_name	last_name	is_alive	age	children	address.city	address.state
0	John	Smith	True	27	Catherine	New York	NY
1	John	Smith	True	27	Thomas	New York	NY
2	John	Smith	True	27	Trevor	New York	NY

Data from the Web

Data from the Web

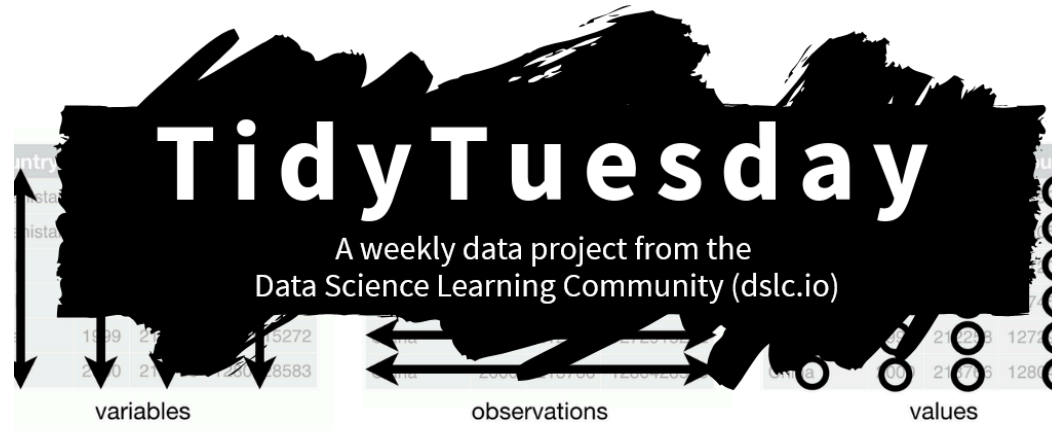
In the simplest scenario, the data you need are ready on the internet in a tabular format.

- Click and download manually.
- Load data from a direct link.

If data are **not** directly downloadable or they update in (near) real-time:

- Fetch the data through web APIs.

TidyTuesday



TidyTuesday is a weekly social data project for the online learning community.

- Data set(s) and a related article every week.
- Emphasize on the understanding of how to summarize and arrange data to make meaningful visualizations.
- Full list of projects can be found on

<https://github.com/rfordatascience/tidytuesday/tree/main>

TidyTuesday: Big Tech Stock Prices

Let's explore the data set on big tech stock prices, posted on Feb 7, 2023.

Here's an overview of the GitHub repository:

<https://github.com/rfordatascience/tidytuesday/tree/main/data/2023/2023-02-07>

Big Tech Stock Prices: Click and Download

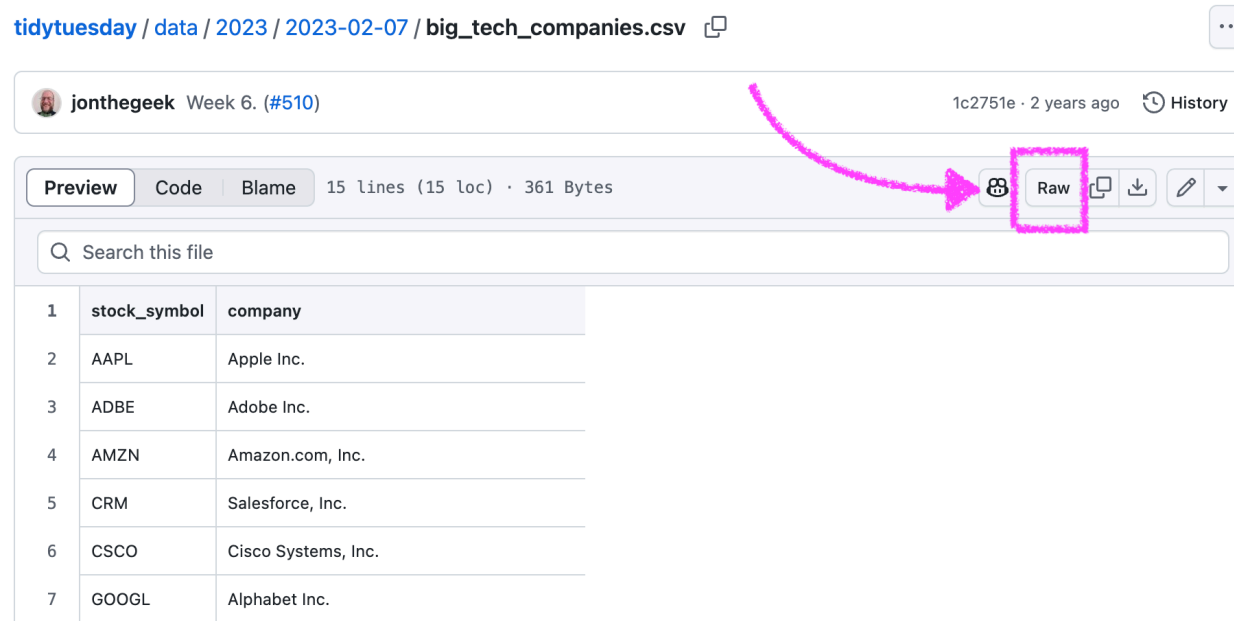
- We can click and download the data sets directly from the link.
- Remember to store the file in `data`.

```
1 tech_companies = pd.read_csv('../data/big_tech_companies.csv',  
2                               header = 0)  
3 tech_companies.head()
```

	stock_symbol	company
0	AAPL	Apple Inc.
1	ADBE	Adobe Inc.
2	AMZN	Amazon.com, Inc.
3	CRM	Salesforce, Inc.
4	CSCO	Cisco Systems, Inc.

Big Tech Stock Prices: Load from Direct Link

- Although we can simply click to download a file, it is often helpful to automate the process.
- Especially when working with data pipelines, reproducible scripts, or dynamic data sets.
- To do so, navigate to the file in the GitHub repository and click on **Raw**.



The screenshot shows a GitHub file page for a CSV file named 'big_tech_companies.csv' in the repository 'tidytuesday / data / 2023 / 2023-02-07'. The file is owned by 'jonthegeek' and is part of 'Week 6. (#510)'. It was last updated '1c2751e · 2 years ago'. The file size is '15 lines (15 loc) · 361 Bytes'. The 'Raw' button is highlighted with a pink box and a pink arrow pointing to it from the file path. Below the file information, there is a search bar and a table of stock data.

	stock_symbol	company
1	AAPL	Apple Inc.
2	ADBE	Adobe Inc.
3	AMZN	Amazon.com, Inc.
4	CRM	Salesforce, Inc.
5	CSCO	Cisco Systems, Inc.
6	GOOGL	Alphabet Inc.

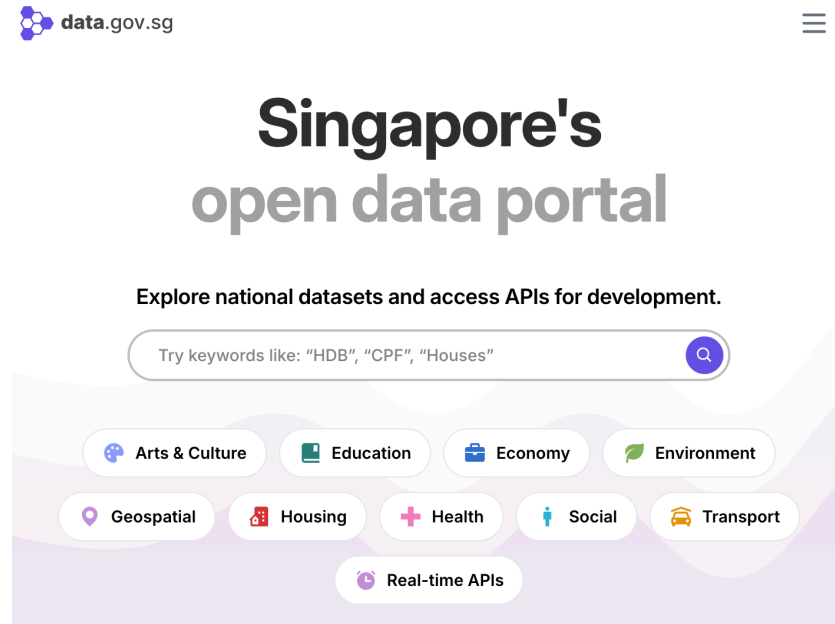
Big Tech Stock Prices: Load from Direct Link

- The **Raw** link points directly to the raw file content:

https://raw.githubusercontent.com/rfordatascience/tidytuesday/refs/heads/main/data/2023/2023-02-07/big_tech_companies.csv

```
1 url = 'https://raw.githubusercontent.com/rfordatascience/tidytuesday/refs/heads/main/data/2023/2023-02-07/big_tech_companies.csv'
2 tech_companies = pd.read_csv(url)
3 tech_companies.head()
```

Singapore's Open Data Portal



data.gov.sg is Singapore's national open data portal launched in 2011.

- 5000+ data sets from government agencies, in the fields of economy, education, environment, housing, public health, etc.
- Data can be accessed and downloaded in various formats.

Example: Employment in Performing Arts

Let's explore and download a static data set on employment in performing arts from 2013 to 2019.

https://data.gov.sg/datasets/d_bcfbc538552950b5800b8455767aae9e/view

The file is stored in CSV format, easily downloadable from the web page.

```
1 path = '../data/EmploymentinArtsandCulturalSectorPerformingArts.csv'  
2 emp = pd.read_csv(path, header = 0)  
3 emp.head(4)
```

	year	type	employment
0	2013	Live Performing Arts	4400
1	2013	Recorded Music	1600
2	2014	Live Performing Arts	5000
3	2014	Recorded Music	1700

Example: Employment in Performing Arts

We can also programmatically query the data.

Here is a sample script on the page:

Sample OpenAPI query

This code can be used to test a sample API query. It retrieves the data catalogue of this dataset. For a complete guide on query parameters and syntax, please refer to the [API documentation](#). Try it out [on your browser](#) to see the response schema.

Python ▾

Copy 

```
import requests

dataset_id = "d_bcfbc538552950b5800b8455767aae9e"
url = "https://data.gov.sg/api/action/datastore_search?resource_id=" + dataset_id

response = requests.get(url)
print(response.json())
```

Example: Employment in Performing Arts

```
1 import requests
2
3 dataset_id = "d_bcfbc538552950b5800b8455767aae9e"
4 url = "https://data.gov.sg/api/action/datastore_search?resource_id=" + dataset_id
5
6 response = requests.get(url)
7 print(response.json())
```

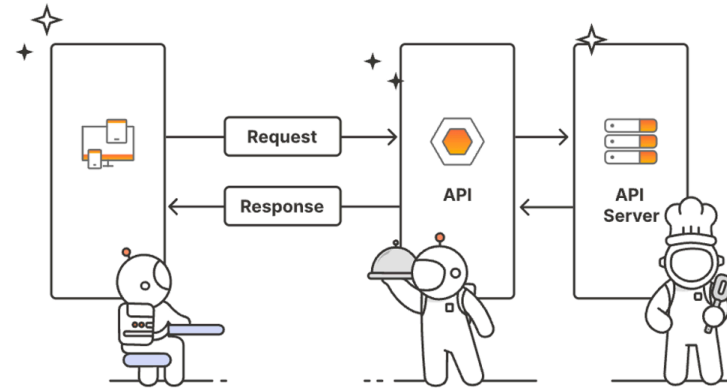
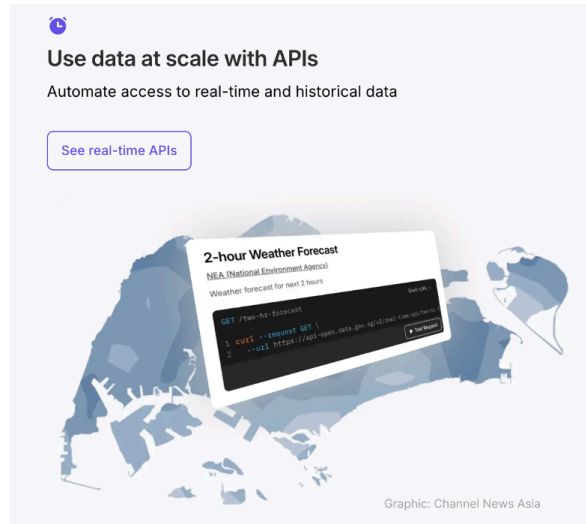
Basic data set info:

- `dataset_id` is a unique ID for each data set on data.gov.sg.
- `url` is the API endpoint to fetch the data set.

Functions from the `requests` package:

- `requests.get()` sends a GET request to the API to retrieve the data.
- `response.json()` parses the response as a JSON object.

More on Web APIs



API stands for **Application Programming Interface**.

- It allows us to automate repetitive, time-consuming work of querying data.
- Can provide additional layers of security against unauthorized breaches by requiring authentication.

Source: Adapted from Postman and data.gov.sg

Example: Daily Traffic Volume

Let’s examine a dynamic data set.

- The Land Transport Authority publishes average daily traffic volume entering the City.
- Available from Jan 2004 and **updates** on a yearly basis.

https://data.gov.sg/datasets/d_3136f317a1f282a33fe7a2f6a907c047/view

Transport

Average Daily Traffic Volume Entering the City

Data from Jan 2004 to Jan 2023 • Updated 2 months ago

[LTA \(Land Transport Authority\)](#)

Daily traffic volume is recorded between 7.30am-7.00pm on weekdays.

Download CSV (303 B)

Subscribe

Data explorer

AverageDailyTrafficVolumeEnteringtheCity.csv (20 rows, 303 B)

Showing 2 of 2 columns ▾

Year Year (YYYY)		Ave Daily Traffic Volume Entering City Text	
(Null)	0.0%	(Null)	0.0%
2014		300400	
2015		297500	

Example: Daily Traffic Volume

- We can pass the `dataset_id` into the API query:

```
1 import requests
2 dataset_id = 'd_3136f317a1f282a33fe7a2f6a907c047'
3 base = 'https://data.gov.sg/api/action/datastore_search?resource_id='
4 url = base + dataset_id
5 response = requests.get(url)
6 response_json = response.json()
```



Example: Daily Traffic Volume [Convert to Data Frame]

The query returns a JSON object with nested structure, which is then converted into a Python dictionary.

- We first explore the structure to see where the actual data are stored:

```
1 response_json.keys()
```

```
dict_keys(['help', 'success', 'result'])
```

```
1 response_json['result'].keys()
```

```
dict_keys(['resource_id', 'fields', 'records', '_links', 'total'])
```

Example: Daily Traffic Volume [Convert to Data Frame]

- The actual data are in `records` as a list of dictionaries.
- The first two records are:

```
1 response_json['result']['records'][0:2]
```

```
[{'_id': 1,  
  'year': '2004',  
  'ave_daily_traffic_volume_entering_city': '246000'},  
 {'_id': 2,  
  'year': '2005',  
  'ave_daily_traffic_volume_entering_city': '244000'}]
```

Example: Daily Traffic Volume [Convert to Data Frame]

- We convert the actual data into a data frame:

```
1 records = response_json['result']['records']  
2 df = pd.DataFrame(records)  
3 df.head(2)
```

	_id	year	ave_daily_traffic_volume_entering_city
0	1	2004	246000
1	2	2005	244000

Example: Daily Traffic Volume [Rename Columns]

In the following, we

- Remove the `_id` column, and
- Rename columns and inspect data types.

```
1 df = df.drop('_id', axis = 1)
2 df.columns = ['year', 'avg_traffic']
3 df.dtypes
```

```
year          str
avg_traffic    str
dtype: object
```

Example: Daily Traffic Volume [Data Types]

Initially, `year` and `avg_traffic` were read in as strings.

We shall convert them to numeric before visualizing the data.

```
1 df['year'] = pd.to_numeric(df['year'])
2 df['avg_traffic'] = pd.to_numeric(df['avg_traffic'])
3 df.dtypes
```

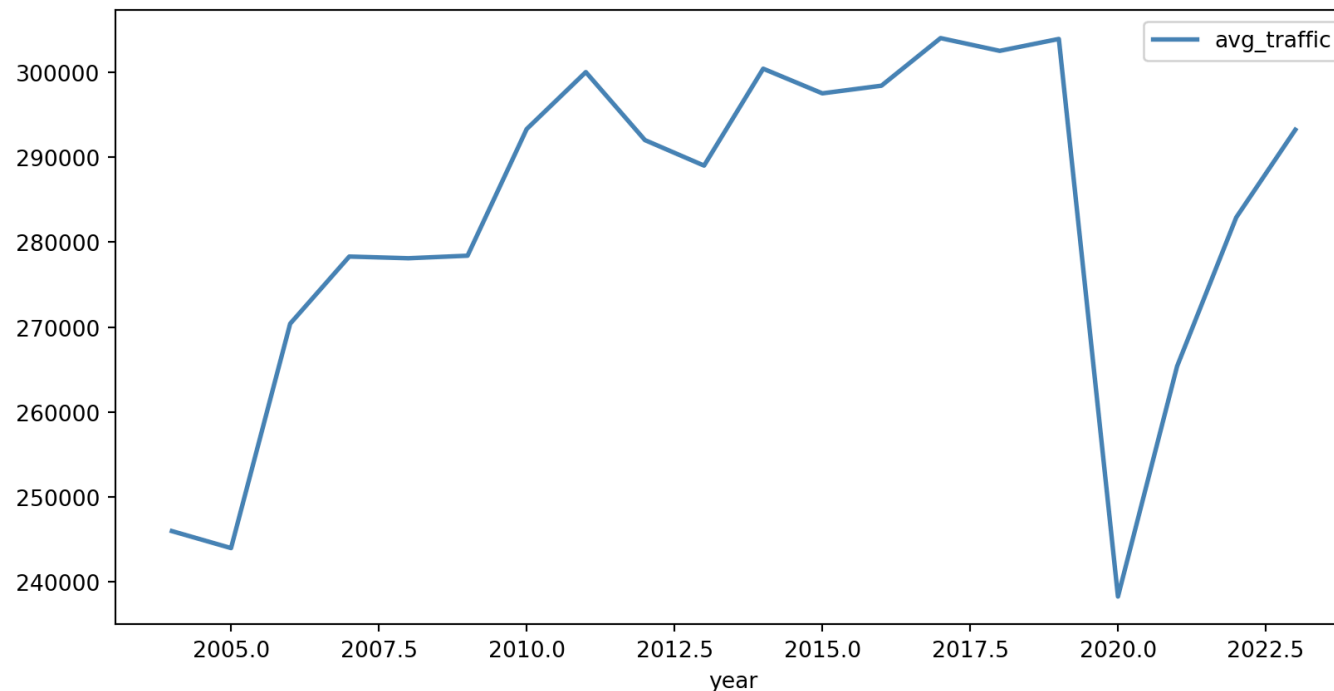
```
year          int64
avg_traffic    int64
dtype: object
```

Example: Daily Traffic Volume [Line Plot]

The plot shows the average daily traffic volume over time. Notice that

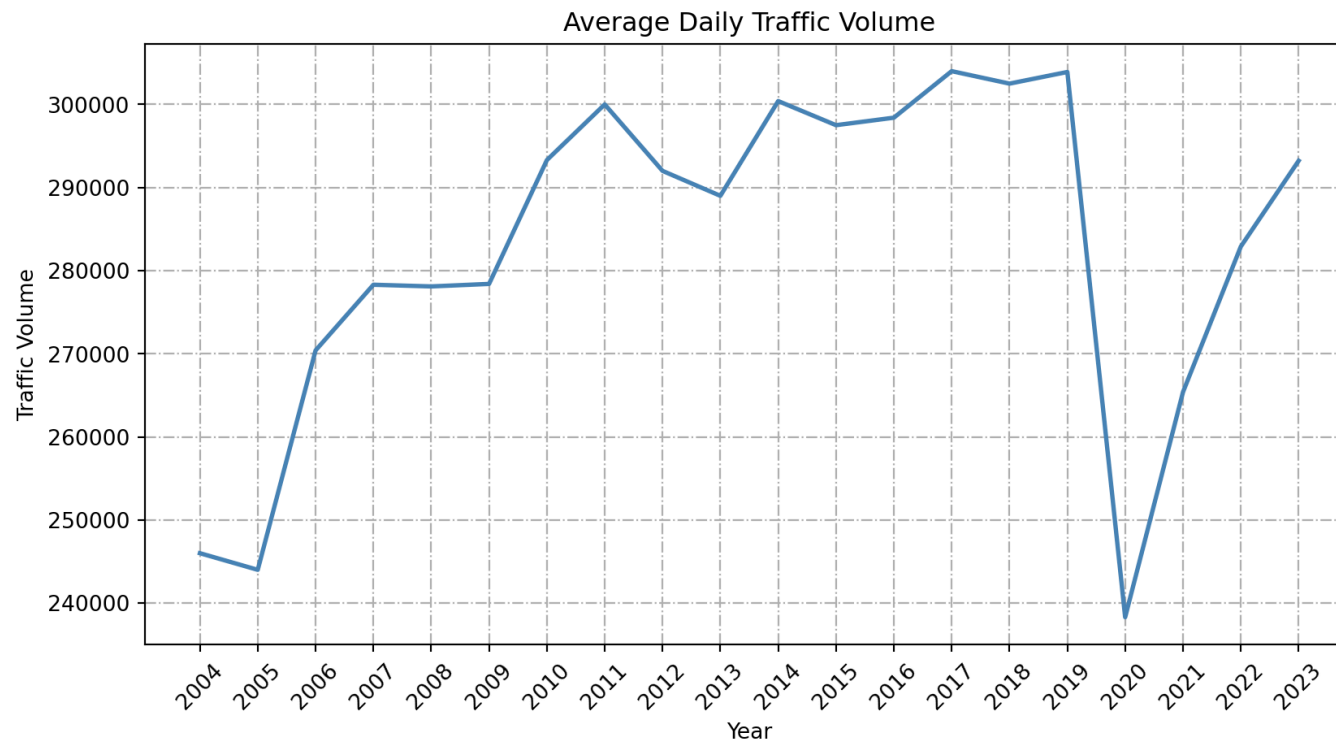
- `year` on the x-axis is treated as floating-point numbers; and
- a legend shows the name of the column we plot (`avg_traffic`).

```
1 df.plot(kind = 'line', x = 'year', y = 'avg_traffic',  
2         linewidth = 2, color = 'steelblue')
```



Example: Daily Traffic Volume [Line Plot]

```
1 df.plot(kind = 'line', x = 'year', y = 'avg_traffic',
2         linewidth = 2, color = 'steelblue', legend = False,
3         title = 'Average Daily Traffic Volume',
4         xlabel = 'Year', ylabel = 'Traffic Volume')
5 plt.xticks(ticks = df['year'], rotation = 45,
6           labels = df['year'].astype(int))
7 plt.grid(axis='both', linestyle = '-.')
8 plt.show()
```

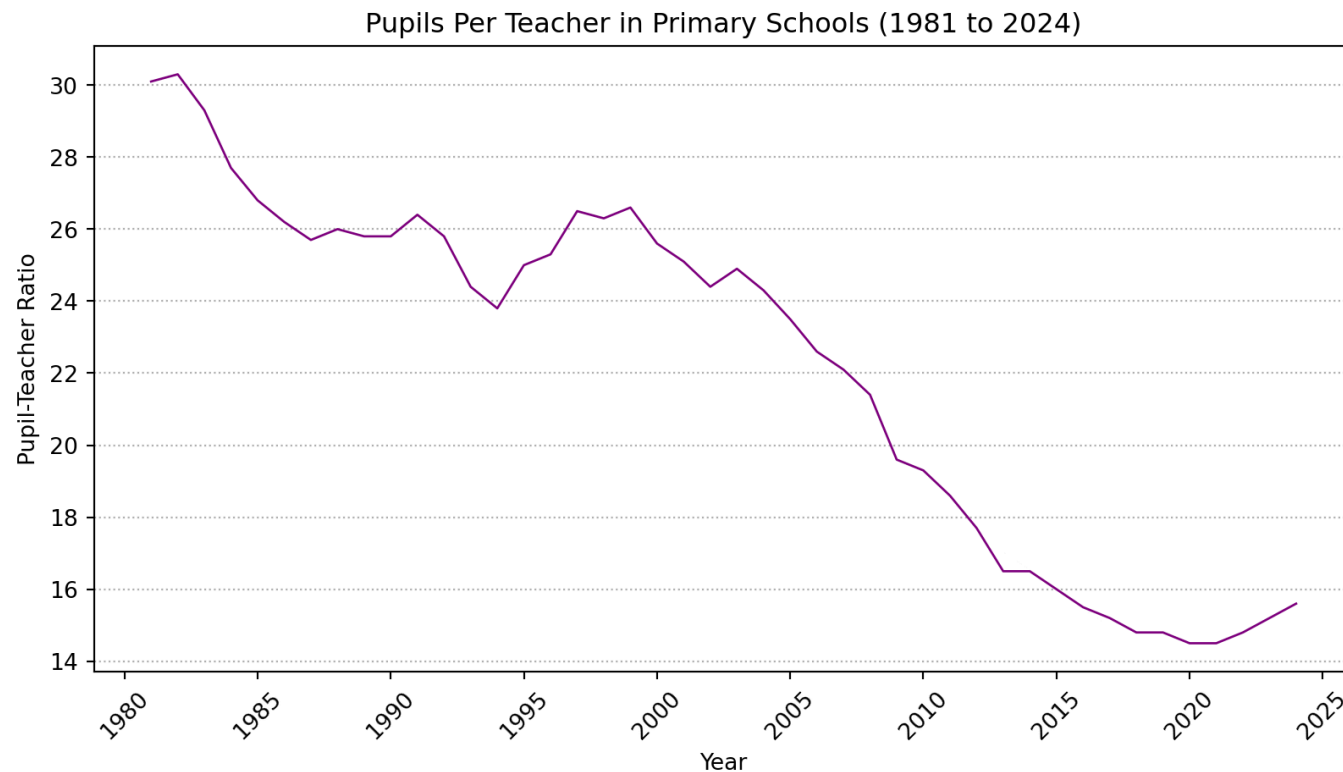


Your Turn! Pupil-Teacher Ratio

There are 5000+ data sets available on data.gov.sg.

In this exercise, practice querying and visualizing data on the Pupil-Teacher Ratio (PTR) in primary schools by year. Try reproducing the plot below.

https://data.gov.sg/datasets/d_bc5a1a10afedf7d25128dae1342c9556/view



Plotting Libraries

Plotting with pandas

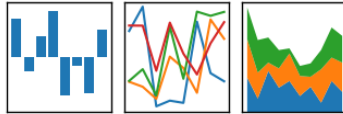
The `pandas` plotting function is built upon `Matplotlib`.

Here are some basic plot types:

	Plot type	When to use
<code>line</code>	Line plot	Show/compare trends over time
<code>bar</code>	Vertical bar chart	Compare categories
<code>barh</code>	Horizontal bar chart	Compare categories with long labels
<code>hist</code>	Histogram	Distribution of a continuous variable
<code>kde</code>	Kernel density	Smooth estimate of a distribution
<code>box</code>	Boxplot	Show/compare spread and outliers (across groups)
<code>scatter</code>	Scatter plot	Relationship between two continuous variables

Plotting Libraries

pandas
 $y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



matplotlib

Both `pandas` and `Matplotlib` are powerful tools for data visualisation in Python.

- Serve different purposes and offer different levels of customization and flexibility.
- `pandas`: Quick and easy plotting directly from DataFrames and Series.
- `Matplotlib`: More flexible and customizable (more on this later).
 - It also offers more plot types.
 - Slightly steeper learning curve.

Plotting Libraries

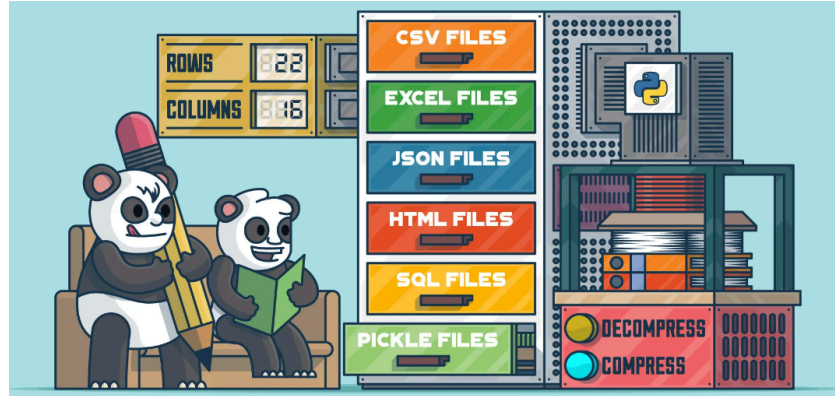


In addition to `pandas` and `Matplotlib`, there are several other popular and powerful visualisation libraries in Python.

- `Seaborn`: Built upon `Matplotlib`.
- `Plotnine`: Implements the Grammar of Graphics (similar to `ggplot2` in R).
- `Plotly`: For interactive visualizations.

There are many more visualization libraries. We will cover some of them in the later weeks.

Summary



This week we learn about importing data from various formats and sources:

1. Plain Text Files
2. CSV Files
3. Excel Files
4. JSON Files
5. Data from the Web

... and a few more ways to analyse and visualise data.